

LangChain, LangGraph & LangChain4j Cheat Sheet

Irurueta Docs · Guides & References · AI · LangChain

Baseline: Python langchain 1.4.3, langchain-core 1.6.5, langgraph 1.2.12, langsmith 0.14.1, langchain-mcp-adapters 0.3.2, langchain-ollama 1.1.0, deepagents 0.7.19 (Python 3.12); npm langchain 1.5.14, @langchain/langgraph 1.4.18; LangChain4j 1.20.2 (agentic / mcp / starters 1.20.2-beta30, experimental), Quarkus extension 1.14.0 (Java 21). Verified 2026-09-29. Quick reference only — verify against the official docs before production use.

PACKAGES → IMPORTS (V1)

Package	Use / import
langchain	langchain.agents, .tools, .messages, .chat_models, .embeddings
langchain-core	Runnable / LCEL, prompts, messages, @tool
langchain-classic	Legacy chains, retrievers, indexes: langchain_classic.*
Partners	langchain_ollama, langchain_openai, langchain_anthropic
langgraph	langgraph.graph, .types, .checkpoint.*, .store.*
langsmith	Tracing, datasets, evaluate
deepagents	create_deep_agent (files, planning, subagents)

uv add langchain langchain-ollama langgraph langsmith

- create_react_agent (langgraph.prebuilt) → langchain.agents.create_agent

CHAT MODELS & MESSAGES

```
from langchain.chat_models import init_chat_model
from langchain.embeddings import init_embeddings
chat = init_chat_model("ollama:llama3.1:8b",
temperature=0)
emb = init_embeddings("ollama:nomic-embed-text")
chat.invoke([{"system","Be terse."}, {"user","hi"}])
for c in chat.stream("hi"): print(c.text)
```

- Missing partner package → ImportError naming it
- msg.content_blocks: provider-agnostic blocks; content stays valid

STRUCTURED OUTPUT

```
class PageMeta(BaseModel):
    title: str
    topics: list[str] = Field(max_length=5)
ex = chat.with_structured_output(PageMeta)
ex.invoke("...") # → PageMeta
# include_raw=True → {"raw","parsed","parsing_error"}

agent = create_agent(model, tools=[t],
response_format=ToolStrategy(PageMeta))
agent.invoke(...)["structured_response"]
```

- Schema: Pydantic, dataclass, TypedDict; JSON Schema needs an explicit strategy
- ProviderStrategy = native; ToolStrategy = extra tool + retries on validation error (Ollama llama3.1 resolved to ToolStrategy)

TOOLS

```
from langchain.tools import tool, ToolRuntime
@tool(parse_docstring=True)
def search_docs(query: str, top_k: int = 4) -> str:
    """Search the docs.
    Args:
        query: what to look for
    """
    @tool("x", args_schema=SearchInput) # explicit schema
    @tool
    def note(text: str, runtime: ToolRuntime) -> str:
        runtime.context / .state / .store / .tool_call_id
```

- Return Command(update=...) to write state; ToolException / handle_tool_errors for failures

CREATE_AGENT + MIDDLEWARE

```
from langchain.agents import create_agent
from langchain.agents.middleware import (
before_model, after_model, wrap_model_call,
wrap_tool_call, dynamic_prompt,
HumanInTheLoopMiddleware,
SummarizationMiddleware)
agent = create_agent("ollama:llama3.1:8b",
tools=[search_docs], system_prompt="You are
DocsAssistant.",
middleware=[...], checkpointer=saver, store=store,
response_format=Schema, context_schema=Ctx)
agent.invoke([{"messages":[{"user","hi"}]},
{"configurable":{"thread_id":"t1"}}])
```

- before_agent/after_agent: once per run; before_model/after_model: each loop turn
- wrap_model_call/wrap_tool_call: around the call (retry, swap model, intercept)
- Built-in: PII, Summarization, Model/Tool Retry, Call Limits, Fallback, TodoList, ContextEditing, LLMTToolSelector
- GraphRecursionError if the loop never ends

LANGGRAPH: STATEGRAPH & REDUCERS

```
from typing import Annotated, TypedDict
import operator
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
class S(TypedDict):
    messages: Annotated[list, add_messages]
    hits: Annotated[list[str], operator.add] # reducer
def answer(s: S) -> dict: return {"hits": ["x"]}
g = StateGraph(S)
g.add_node("answer", answer)
g.add_edge(START, "answer"); g.add_edge("answer", END)
g.add_conditional_edges("answer", route)
app = g.compile(checkpointer=saver)
```

- No reducer = overwrite; reducer = merge concurrent writes
- Command(update=..., goto="n") routes from a node; Send("n", payload) fans out; MessagesState shortcut

CHECKPOINTER & STORE

```
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.checkpoint.postgres import PostgresSaver
from langgraph.store.memory import InMemoryStore
cfg = {"configurable": {"thread_id": "alice-1"}}
with SqliteSaver.from_conn_string("db.sqlite") as saver:
    agent = create_agent(m, tools=[],
checkpointer=saver)
store = InMemoryStore(index={"embed": emb,
"dims": 768, "fields": [{"text":}]})
store.put({"user","42"}, "k", {"text": "..."})
store.search({"user","42"}, query="...")
```

- Checkpoint = one thread's state; store = memory across threads (namespaces)
- No thread_id → ValueError; get_state / update_state
- Separate installs: langgraph-checkpoint-sqlite, -postgres

HUMAN-IN-THE-LOOP

```
from langgraph.types import interrupt, Command
def approval(s):
    ok = interrupt({"question": "File issue?"})
    return Command(goto="file" if ok else "end")
g.invoke(inputs, cfg) # pauses
g.invoke(Command(resume=True), cfg) # same thread_id

HumanInTheLoopMiddleware(interrupt_on={
"open_issue": {"allowed_decisions":
["approve","edit","reject"]})
agent.invoke(Command(resume=["decisions":
[{"type":"approve"}]}), cfg)
```

- Needs checkpointer + thread_id + JSON payload; node re-runs from its start on resume
- Time travel: get_state_history, replay / fork

STREAMING

```
for part in agent.stream(inp, stream_mode=
"messages","custom",
version="v2"): ...
from langgraph.config import get_stream_writer
get_stream_writer()({"progress": "3 hits"})
async for e in agent.astream_events(inp, version="v2"): ...
```

Mode	Emits
values	full state per step
updates	node deltas by node name
messages	(token chunk, metadata)
custom	your get_stream_writer() data

- Also checkpoints, tasks, debug; SSE via FastAPI EventSourceResponse

MCP ADAPTER

```
from langchain_mcp_adapters.client import
MultiServerMCPClient
client = MultiServerMCPClient({
"docs": {"transport":"stdio","command":"python",
"args":["docs_server.py"]},
"web": {"transport":"streamable_http",
"url":"http://127.0.0.1:8765/mcp"}},
tool_name_prefix=True)
tools = await client.get_tools() # BaseTool list
agent = create_agent(model, tools=tools)
# newer: from langchain.mcp import MCPAdapter
# async with MCPAdapter(config) as a: ...
```

- One session per call by default; client.session() + load_mcp_tools for stateful servers

LANGSMITH: TRACING & EVALUATION

```
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY=... # from a secret store
export LANGSMITH_PROJECT=docsassistant-dev
# LANGSMITH_ENDPOINT for non-US regions
# legacy: LANGCHAIN_TRACING_V2 / LANGCHAIN_API_KEY
from langsmith import evaluate, traceable
from openevals.llm import create_llm_as_judge
from agentevals.trajectory_match import \
create_trajectory_match_evaluator
def correct(outputs, reference_outputs): ...
evaluate(target, data="docs-q*", evaluators=[correct],
experiment_prefix="v2")
```

- Trajectory modes: strict, unordered, subset, superset
- pytest: @pytest.mark.langsmith

LANGGRAPH CLI & DEPLOYMENT

```
{"dependencies": ["."],
"graphs": {"docs": "./agent.py:agent"},
"env": ".env"} # langgraph.json
langgraph dev --no-browser --port 2247 # no Docker
langgraph validate | dockerfile | build | up
langgraph deploy # LangSmith-hosted
from langgraph_sdk import get_client
```

- Agent Server: assistants, threads, runs, crons
- Renamed: LangGraph Platform → LangSmith Deployment; Studio → LangSmith Studio

LCEL, PROMPTS & RAG BRIDGE

```
chain = prompt | chat | StrOutputParser()
chain.invoke({"q": "..."}); chain.batch([...])
retriever = vs.as_retriever(search_kwargs={"k": 4})
@tool
def search_docs(q: str) -> str:
    return "\n".join(d.page_content for d in
retriever.invoke(q))
```

- LCEL for fixed pipelines; create_agent when the model decides; StateGraph for explicit control

LANGCHAIN4J (JAVA) EQUIVALENTS

```
// import langchain4j-bom 1.20.2 (langchain4j, -
ollama);
// -agentic / -mcp / starters: 1.20.2-beta30
interface DocsAssistant {
@SystemMessage("You are DocsAssistant.")
String chat(@MemoryId String id, @UserMessage String q);
} // @MemoryId ~ thread_id
ChatModel model = OllamaChatModel.builder()
.baseUrl("http://localhost:11434")
.modelName("llama3.1:8b").build();
DocsAssistant a =
AiServices.builder(DocsAssistant.class)
.chatModel(model)
.tools(new DocsTools()) // @Tool("Search docs") ~
@tool
.chatMemoryProvider(id ->
MessageWindowChatMemory.withMaxMessages(10))
.build();
// Agentic (experimental):
AgenticServices.agentBuilder(X.class),
// @Agent(outputKey="notes"), sequenceBuilder /
loopBuilder /
// parallelBuilder / supervisorBuilder
// MCP: McpToolProvider(DefaultMcpClient + Stdio- or
StreamableHttpMcpTransport) -> .toolProvider(p)
```

- Structured output: return a POJO from the method; streaming: TokenStream / Flow.Publisher; guardrails: InputGuardrail / OutputGuardrail; Spring Boot starter and quarkus-langchain4j

JAVASCRIPT EQUIVALENTS

```
npm i langchain @langchain/core @langchain/langgraph
@langchain/ollama zod
import { createAgent, tool, toolStrategy } from
"langchain";
import { MemorySaver, StateGraph, Command, interrupt }
from "@langchain/langgraph";
const t = tool(fn, { name:"search_docs",
schema: z.object({query: z.string()}) });
const agent = createAgent({ model:
"ollama:llama3.1:8b",
tools:[t], systemPrompt:"...",
checkpointer: new MemorySaver(),
responseFormat: toolStrategy(Schema) });
await agent.invoke({messages:[...],
{configurable:{thread_id:"t1"}}});
```

- Middleware: createMiddleware, humanInTheLoopMiddleware; React: useStream

PITFALLS & CHECKLIST

- Old imports (langchain.chains) → langchain_classic
- Always set a thread_id and derive it server-side (it is the only access key)
- Bound the loop: recursion limit, call-limit middleware
- Gate write tools with human approval
- Never put API keys in source; mask PII in traces
- Agentic module is beta: pin versions
- Python and npm version numbers differ